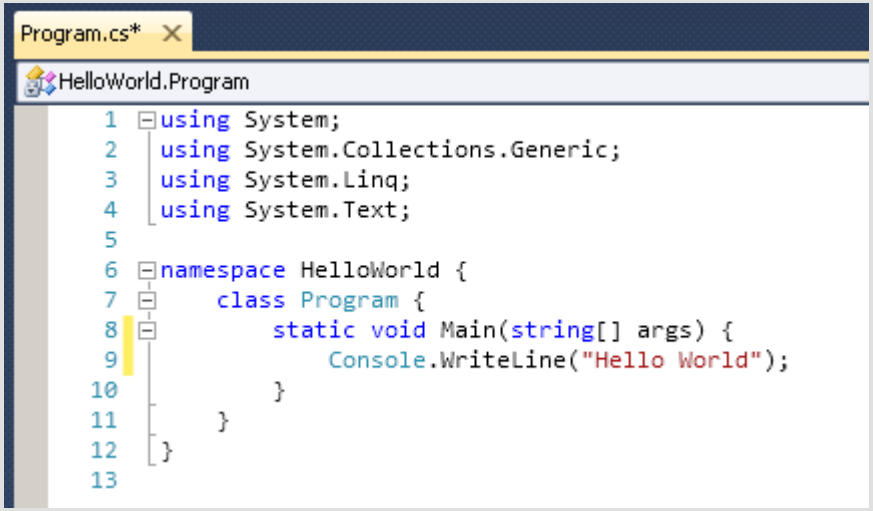
NESNEYE DAYALI PROGRAMLAMA

Ders 1: C# Temel Kavramlar

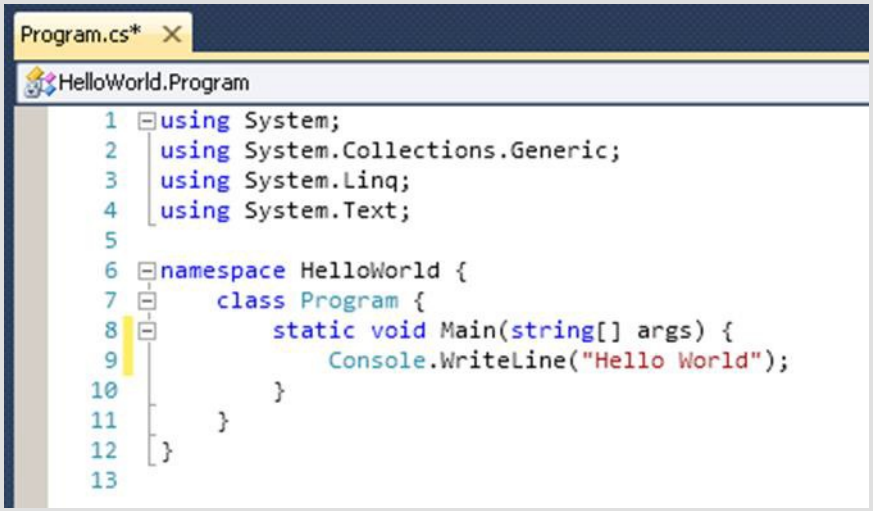
Merhaba C#

- Sınıf bilgisi
  - using
- Main fonksiyonu
- Console
  - Konsol uygulamaları için standart giriş ve çıkış hata akışlarına temsil eder.
- Açıklama satırı yazma
  - // ve /* … */



Using

- Using kelimesi sıklıkla isim uzayları (namespace) içerisindeki sınıfların kod içerisinde kolayca kullanılabilmesini sağlamak amacıyla kullanılır.

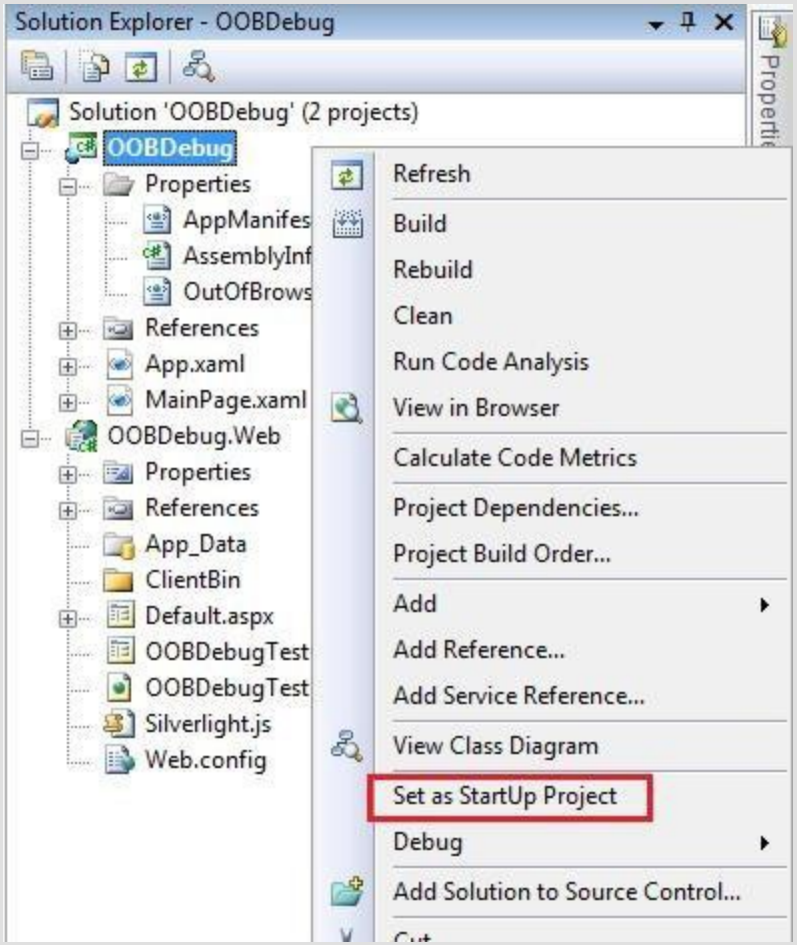


Command Ekranından Derlenmiş Kodun Çalıştırılması

- İşletim sisteminin RUN bölümüne cmd yazıp veya Donatılar bölümünden console uygulamasını bulup çalıştırın.
- Programınızın kaydedildiği yerde Debug ve Bin dizinini gidin.
  - cd.. Üst dizine geçmek.
  - cd dizin_ismi: belirlenen dizine gitmek
  - dir: dizin içindeki dizin ve dosyaları görüntülemek
  - Bin dizinine ulaşınca exe uzantılı dosyanın ismini yazıp enter'a basın.

Birden fazla proje ile çalışma

- Solution Explorer'dan tüm projelerinizi görebilirsiniz.
- Solution üzerine gelip farenin sağ tuşu ile yeni bir proje eklenebilir. (Add->New Project)
- Proje üstüne gidip «Set as Startup Project» ile çalışacak proje belirlenir.
- Derste her yazılan kod için bir proje oluşturulacaktır. Evde çalışırken seçili projeyi değiştirmeniz gerektiğini unutmayın.



Veri Türleri ve Değişken

- Değişken, programın çalışması için gerekli verilerin tanımlanarak, bellek üzerinde tutulduğu bölgelere verilen isimlerdir.
- Tanımlama o bellek bölgesinde tutulacak olan verinin türünün belirtilmesidir.
- Bir değişken tanımlandıktan sonra aynı türden değer atamak koşuluyla değeri değiştirilebilir.



Veri Türleri

Veri Türü	CTS Karşılığı	Açıklama	Max ve Min aralık yada değeri
sbyte	**System.Byte**	8 bit işaretli tamsayı	-128 : 127
short	**System.Int16**	16 bit işaretli tamsayı	-32.768 : 32.767
int	**System.Int32**	32 bit işaretli tamsayı	-2.147.483.648 : 2.147.483.647
long	**System.Int64**	64 bit işaretli tamsayı	-9.223.372.036.854.775.808 : -9.223.372.036.854.775.807
byte	**System.Byte**	8 bit işaretsiz tamsayı	0 : 255
ushort	**System.UInt16**	16 bit işaretsiz tamsayı	0 : 65.535
uint	**System.UInt32**	32 bit işaretsiz tamsayı	0 : 4.294.967.295
ulong	**System.UInt64**	64 bit işaretsiz tamsayı	0 : 18.446.744.073.709.551.615
float	**System.Single**	32 bit tek kayan sayı	+yada - $1,5*10^{-45}$: + ya da - $3,4*10^{38}$
double	**Sytem.Double**	64 bit çift kayan sayı	+yada - $5*10^{-324}$: + ya da - $1,7*10^{308}$
decimal	**System.Decimal**	128 bit ondalıklı sayı	+yada - $1,5*10^{-28}$: + ya da - $7,9*10^{28}$
bool	**System.Boolean**		true ya da false
char	**System.Char**	Karakterleri temsil eder	16 Unicode karakterleri

object	**System.Object**	Bütün veri türlerinin türediği kök eleman
string	**System.String**	Unicode karakterlerinden oluşan string

Değişken Tanımlama

<DeğişkenTürü> <DeğişkenAdı>;

<DeğişkenAdı> = <DeğikenDeğeri>;

//veya

<DeğişkenTürü> <DeğişkenAdi> = <DeğişkenDeğeri>;

Değişken Tanımlama Örnekleri

- int sayi = 5;
- int x, y = 8, z; //Aynı türdeki değişkenler aynı anda tanımlanabilir.
- bool aktif = true;
- float f = 5.4f; // Değerin sonuna eklediğimiz f harfi değişkenin float türünde olduğunu gösterir.
- double d = 3.2;
- long l = 123456789;
- short s = -312;
- decimal dec = -5.26m; //Değerin sonundaki m harfi değişkenin decimal türünde olduğunu gösterir.
- char ch = 'c'; //Char tipinde ki değişkenler tek tırnak içerisine yazılır.
- string deger = "merhaba";

Const ve Readonly

- Uygulama çalışırken değeri değiştirilemeyecek değişken tanımlamamızı sağlarlar.
- Const (Sabit): Class seviyesinde tanımlanır ve tanımlanma anında değeri verilmek zorundadır. Sonradan değeri değiştirilemez.
- Readonly (Sadece-Okunabilir): Class seviyesinde tanımlanır. Tanımlandığı anda değeri verilebilir veya Class Constructor'ında değeri verilebilir. Sonradan değeri değiştirilemez.

Veri Türleri Arasında Dönüşüm

- X = Y (bilinçsiz dönüşüm-implicit type conversion)
- X = (int) Y (bilinçli dönüşüm-explicit type conversion)
- X = System.Convert.ToString(Y);

Bilinçsiz Dönüşüm (implicit type conversion)

- C#'ta düşük kapasiteli bir değişken, sabit ya da değişken ve sabitlerden oluşan matematiksel ifade daha yüksek kapasiteli bir değişkene atanabilir. Buna bilinçsiz tür dönüşümü denir, bunun için herhangi bir özel kod gerekmez.

```
using System;
class TurDonusumu
{
        static void Main()
        {
                byte a=5;
                short b=10;
                sbyte c=30;
                int d=a+b+c;
                string e="deneme";
                char f='k';
                object g=e+f+d;
                long h=d;
                float i=h;
                double j=i;
                double k=12.5f;
                Console.WriteLine(j+k);
        }
}
```

```
using System;
class TurDonusumu
{
        static void Main()
        {
                char a='h';
                int b=a;
                Console.WriteLine(b);
        }
}
```

```
using System;
class TurDonusumu
{
        static void Main()
        {
                byte a=5, b=10;
                short d=2, e=9;
                byte f=a+b;
                short g=d+e;
                Console.WriteLine(f+g);
        }
}
```

Bilinçli Dönüşüm (explicit type conversion)

- Bilinçli tür dönüşümü genellikle derleyicinin izin vermediği durumlarda kullanılır.
- Bilinçli tür dönüşümüyle küçük türün büyük türe dönüştürülmesi sağlanabilse de aslında bu gereksizdir, çünkü aynı şeyi bilinçsiz tür dönüşümüyle de yapabilirdik.

```
using System;
class TurDonusumu
{
        static void Main()
        {
                int a=5;
                byte b=(byte)a;
                Console.WriteLine(b);
        }
}
```

```
using System;
class TurDonusumu
{
        static void Main()
        {
                byte b=(byte)12.5f;
                Console.WriteLine(b);
        }
}
```

Bilinçli Dönüşüm

- object türündeki bir değişkene başka herhangi bir türdeki değişken ya da sabit (string dışında) + işaretiyle eklenemez.

```
using System;
class TurDonusumu
{
        static void Main()
        {
                object a='5';
                int b=4;
                Console.WriteLine(a+b);
        }
}
```

- Hatalı Durum

Bilinçli Dönüşüm

- object türündeki bir değişkene herhangi bir türdeki değişken ya da sabit atanabilir (bilinçsiz tür dönüşümü). Ancak object türündeki herhangi bir değişkeni başka bir türe çevirmek için tür dönüştürme işlemi kullanılır. Ayrıca da dönüştürülen türlerin uyumlu olması gerekir. Yani object türüne nasıl değer atandığı önemlidir.

```
using System;
class TurDonusumu
{
        static void Main()
        {
                object a="5";
                int b=(int)a;
                Console.WriteLine(b);
        }
}
```

- Hatalı Durum

Bilinçli Dönüşüm

- object türündeki bir değişkene herhangi bir türdeki değişken ya da sabit atanabilir (bilinçsiz tür dönüşümü). Ancak object türündeki herhangi bir değişkeni başka bir türe çevirmek için tür dönüştürme işlemi kullanılır. Ayrıca da dönüştürülen türlerin uyumlu olması gerekir. Yani object türüne nasıl değer atandığı önemlidir.

  - Hatalı Durum

```
using System;
class TurDonusumu
{
        static void Main()
        {
                object a="5";
                int b=(int)a;
                Console.WriteLine(b);
        }
}
```

  - Doğru Kullanım

```
using System;
class TurDonusumu
{
        static void Main()
        {
                object a='k';
                char b=(char)a;
                Console.WriteLine(b);
        }
}
```

Bilinçli Dönüşüm

- string türüyle ilgili dönüşümler

```
using System;
class TurDonusumu
{
        static void Main()
        {
                int a=6;
                string b=a.ToString();
                Console.WriteLine(b);
        }
}
```

Tür Dönüşüm

- System isim alanının altındaki Convert sınıfı içinde tür dönüşümü yapabileceğimiz birçok metot bulunur. Bu metotlarla hemen hemen her türü her türe dönüştürebiliriz. Ancak bunun için değişken türlerinin CTS karşılıklarını bilmeliyiz. Değişken türleri ve CTS karşılıkları aşağıdaki tabloda listelenmiştir.

Tür	CTS karşılığı
bool	Boolean
byte	Byte
sbyte	Sbyte
short	Int16
ushort	UInt16
int	Int32
uint	UInt32
long	Int64
ulong	UInt64
float	Single
double	Double
decimal	Decimal
char	Char

```
Convert.ToBoolean(x)
Convert.ToByte(x)
Convert.ToSbyte(x)
Convert.ToInt16(x)
Convert.ToUInt16(x)
Convert.ToInt32(x)
Convert.ToUInt32(x)
Convert.ToInt64(x)
Convert.ToUInt64(x)
Convert.ToSingle(x)
Convert.ToDouble(x)
Convert.ToDecimal(x)
Convert.ToChar(x)
```

Bir string bir integer dönüştürme yöntemleri

- Int32.Parse(string s)
- Convert.ToInt32(string s)
- Int32.TryParse(string s, out int result)

Int32.Parse

- Int32.Parse metodunun 3 adet Exception'ı bulunur. Bunlar ArgumentNullException, FormatException ve OverflowException'dır.
  - ArgumentNullException: Giriş değeri null olduğu zaman gerçekleşir.
  - FormatException: Giriş değeri doğru formatta olmadığı zaman gerçekleşir. Örneğin; alfanümerik bir değer gibi.
  - OverflowException: Giriş değeri Int32 sınırlarını aştığında gerçekleşir.
- Try catch ile exception durumlar yakalanabilir. Try catch ileri ki haftalarda anlatılacaktır.

Convert.ToInt32

- Convert.ToInt32 metodunun Int32.Parse'tan en önemli farkı, yalnızca string verileri Int32 türüne çevirmek değil, bunların dışında bool, byte, char,DateTime, decimal, double, float, int, long, object, sbyte, short, string, uint, long ve short veri tiplerine de çevirebiliyor olması geliyor.
- İkinci önemli fark ise Int32.Parse'ın aksine 3 exception yerine 2 exception'a sahip olması. Bunlar FormatException ve OverflowException'dır. Fark ettiğiniz üzere ArgumentNullException Convert.ToInt32 bulunmuyor. Sebebi ise giriş verisinin null olması durumunda hata fırlatmak yerine geriye 0 değeri döndürmesinden kaynaklanıyor.

Int32.TryParse

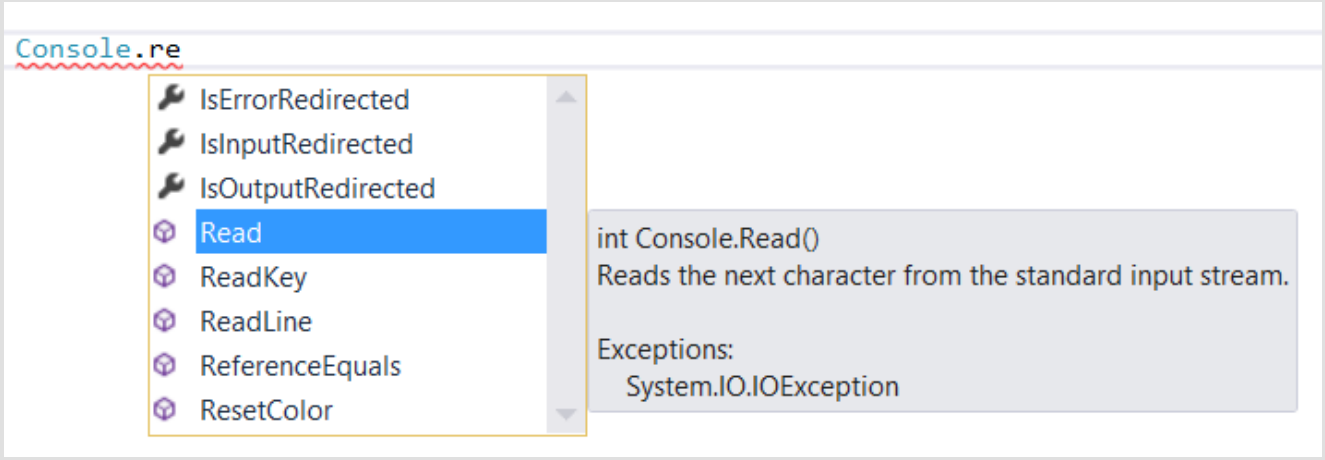
- Int32.TryParse; işleme sokulacak olan verinin, tipini sorgulayarak bizlere bool yani mantıksal bir ifade geri döndürür.
- Int32.Parse ve Convert.ToInt32'de belirttiğimiz exceptionların tümü Int32.TryParse'ta toplu halde bulunur. Yani convert yapacağınız verinin null, doğru formatta olup olmaması ya da Int32 sınırlarını aşması gibi durumlarda geriye bool veri tipinde false değeri döndürür. Ancak bunu int veri tipinde saklar ve dolayısı ile görünen sonuç "0" olarak gönderilir.

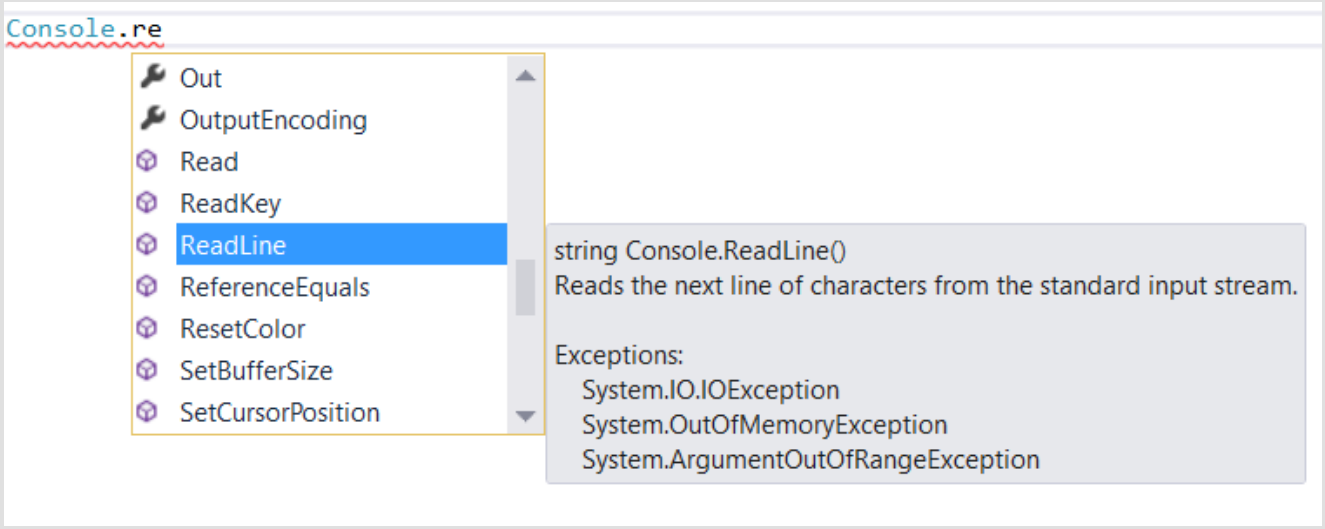
Console Write ve WriteLine

- Console ekranına çıktı için kullanılır.
- Write işleminden sonra alt satıra geçilmez. C'deki gibi \n ile alt satıra geçilebilir.
- WriteLine işleminden sonra alt satıra geçilir.

Console Read - ReadLine

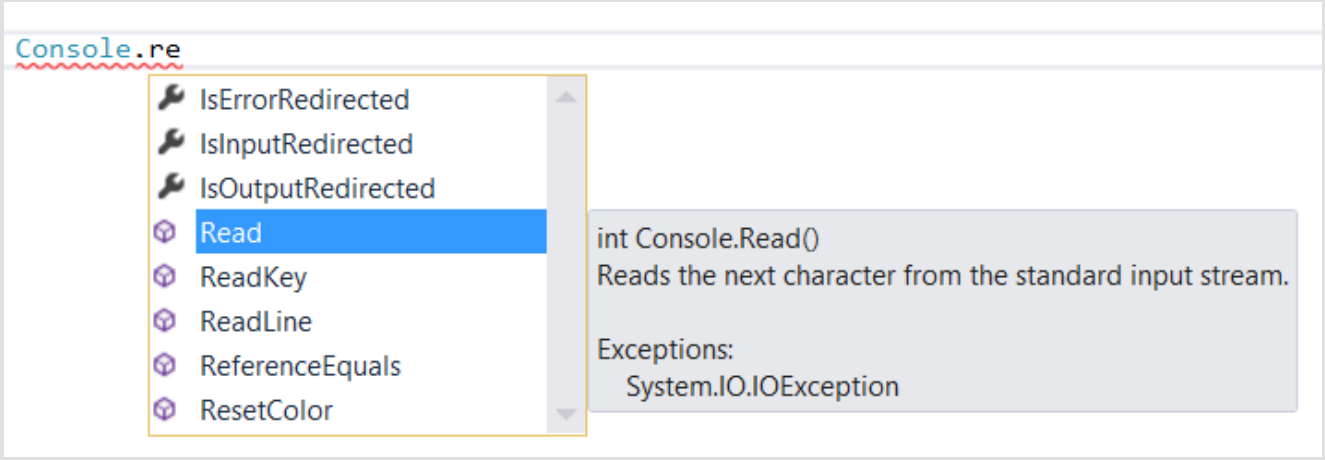
- Console yazıp noktaya tıklayın ve fonksiyonların döndürdüğü değerleri bakın.

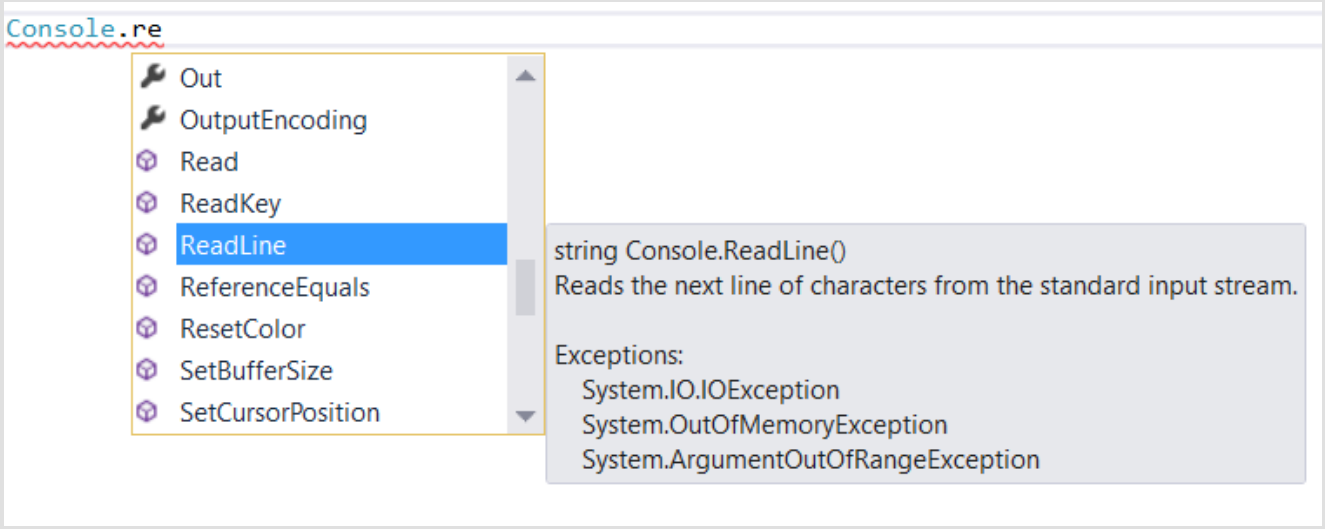




Console Read - ReadLine

- Read int türünden ve Readline'ın string türünden olduğunu dikkat edin.





ReadLine örneği

```
using System;
class TurDonusumu
{
        static void Main()
        {
                string s1, s2;
                int sayi1, sayi2;
                int Toplam;
                Console.Write("Birinci sayıyı girin: ");
                s1=Console.ReadLine();
                Console.Write("İkinci sayıyı girin: ");
                s2=Console.ReadLine();
                sayi1=Convert.ToInt32(s1);
                sayi2=Convert.ToInt32(s2);
                Toplam=sayi1+sayi2;
                Console.Write("Toplam= "+Toplam);
        }
}
```

İfadeler

İfade, bir değeri hesaplayacak matematiksel bir formüldür

```
(a+b)/4;
a*b+c;
```

İfadeler Operatörler ile biçimlendirilirler

İfadeler ?

+,-,*,/,%... gibi matematiksel operatörlerin kullanıldığı, bir değeri hesaplamak için kullanılan matematiksel formüllerdir

```
santigrat = (fahrenheit - 32) / 1.8;
alan   = 3.14 * yaricap * yaricap;
cevre  = 2 * 3.14 * yaricap;
hacim  = uzunluk * genislik * yukseklik;
toplam = toplam + 5;
…
```

Operatörler

C# operatörleri aşağıdakiler gibi sınıflandırılabilir.

- Atama Operatörü (=)
- Aritmetik Operatörler (+, -, *, /, %)
- Tekli Aritmetik Operatörler (++, --)
- Karşılaştırma operatörleri (<, <=, ==, >=, >)
- Mantıksal operatörler (&&, ||, !)
- Birleşik atama operatörleri (+=, -=, *=, …)

Aritmetik Operatörler

Temel operatörler aritmetik işlemler için kullanılan operatörlerdir. Genelde tüm programlama dilleri için aynıdır.

+	Toplama
-	Çıkarma
*	Çarpma
/	Bölme
%	Mod alma (kalan verme)

Binary operatörler
(iki operanda uygulanır)

++	1 Artırma
--	1 Eksiltme

Unary operatörler
(bir tek operanda uygulanır)

Aritmetik Operatörler: Örnek

İşlem	Operator	denklem	a'nın önceki değeri	a'nın sonraki değeri
Toplama	+	a = a + 2	4	6
Çıkarma	-	a = a – 2	4	2
Çarpma	*	a = a * 2	4	8
Bölme	/	a = a / 2	4	2
Modül	%	a = a % 3	4	1

İşlem	Operator	denklem	a'nın önceki değeri	a'nın sonraki değeri
Artırma	++	++a	4	5
Eksiltme	--	--a	4	3

Operator - Notlar

Eğer iki operant int ise, bölme (/) işlemi tam bölme yapar.

```
int a = 5;
int b = 4;
int sonuç = a / b; /* 1.25 -> 1 */
Console.Write("sonuç: {0}", sonuç);
/* ekrana 1 yazar*/
```

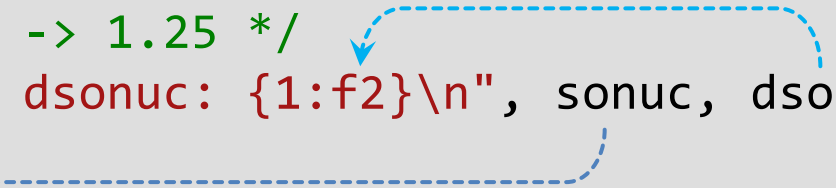


```
sonuç: 1
```

Karışık Operandlar

Eğer operandlardan biri float/double ise diğer operand işlemden önce otomatik olarak float/double a dönüştürülür.

```
int sonuc;
double dsonuc;
sonuc = 5 / 4;      /* 1.25 -> 1 */
dsonuc = 5 / 4.0;   /* 1.25 -> 1.25 */
Console.Write("sonuc: {0}, dsonuc: {1:f2}\n", sonuc, dsonuc);
```



```
sonuc: 1, dsonuc: 1.25
```

Atama Operatörü

Bir değeri bir değişkene atar
C# de = ile gösterililr.
Atama operatörü kullanım şekli:

```
değişken = ifade;
```

Örnekler :

```
x = 17;
alan = kenar * kenar;
toplam = toplam + data;
```

Atama operatörü

Her zaman sağ tarafı sol tarafa atar.

Atama operatörünün sol tarafı;

Değiştirilebilir,
Değer atanabilir,

Çağrılabilir olmalıdır.

```
alan = 45;
benim_uzun_degiskenim_123 = a+b+c;
```

- Aşağıdakiler birer atama ifadesi olamazlar!

```
25 = ...;      // error
b - 5 = ...;   // error
(x) = ...;     // error
n * 2 = ...;   // error
```

Çoklu Atamalar

Bir satırda birden fazla atama yapılabilir

Fakat kodun anlaşılırlığını zorlaştırabilir!

```
int a = 5, b = 6, c; // bir tanesini boş geçiyoruz
```

```
int a, b, c;
a = b = c = 6;  // OK. kullanılabilir
```

```
int a = b = 3;  // Syntax error, b tanımlanmamış
```

Tekli Operatörler

`sonuc = ++a;`

önce arttırılır, sonra atanır

Aşağıdakine denktir:

```
a = a+1;
sonuc = a;
```

- **`sonuc = a++;`**
  - önce atanır, sonra arttırılır
  - Aşağıdakine denktir:

```
sonuc = a;
a = a+1;
```

- **`sonuc = --a;`**
  - önce eksiltilir, sonra atanır
  - Aşağıdakine denktir:

```
a = a-1;
sonuc = a;
```

- **`sonuc = a--;`**
  - önce atanır, sonra eksiltilir
  - Aşağıdakine denktir:

```
sonuc = a;
a = a-1;
```

- Tekli operatörler int, float, double gibi sayısal değerli operandların tümü ile kullanılabilir.

Birleşik Atama Operatörleri

Birleşik atama operatörleri:

+= -= *= /= %= …

Örneğin:

```
toplam += data;          /* toplam = toplam + data; */
a -= 6;                  /* a = a – 6; */
carpim *= data;          /* carpim = carpim*data; */
a %= 2;                  /* a = a % 2; */
a /= b+c;                /* a = a / (b + c); */
…
```

Birleşme & Öncelik

Bir ifade birden fazla aynı tip operatör içeriyorsa **birleşme özelliği** uygulanır.

```
sonuc = a+b+c; → sonuc = (a+b)+c;   /* soldan birleşme */

a = b = c;          → a = (b=c);            /* sağdan birleşme */
```

- Eğer bir ifade birbirinden farklı operatörler içeriyorsa , **öncelik** kuralı uygulanır.

```
sonuc = a+b*c; → sonuc = a+(b*c);   /*  '*' çarpma '+' toplamadan */
                                    /*  daha  önceliklidir.       */
```

Öncelik Kuralları

işaret olan - örneğin:
x = -5;

Öncelik	Operator	
1	- , ++, --	
2	*, /, %	
3	+, -	
4	=, +=, -=, ..	

- Bütün öncelik kurallarını hatırlamak zordur.
- Bu yüzden ifadeleri parantez ile ayırmak en iyi yöntemdir.
  - sonuc = (a*b) + (a/b);

Karşılaştırma Operatörleri

Karşılaştırma operatörleri iki değeri karşılaştırır ve ilgili operatöre göre doğru veya yanlış; true veya false olduğuna karar verir.

Operator	Anlam
==	Eşittir
!=	Eşit değil
<	Küçüktür
<=	Küçük eşit
>	Büyüktür
>=	Büyük eşit

Karşılaştırma operatörleri Örnek-1

a = 1, b = 2, ve c = 3 olduğunu varsayalım.

İfade	Karar
a<b	true
(a+b) >= c	true
(a+b) >= (a+5)	false
c!=3	false
b==2	true

Karşılaştırma operatörleri Örnek-2

```
int sicaklik = 50;
double yagmur = 0.75;

Console.WriteLine("sıcak mı?: " + (sicaklik >= 70));
Console.WriteLine("bulutlu mu?: " + (sicaklik <= 50));
Console.WriteLine("mükemmel mi?: " + (sicaklik == 80));
Console.WriteLine("yağabilir mi?: " + (yagmur > 0.6));
Console.WriteLine("kuru mu?: " + (yagmur < 0.3));
```

```
sıcak mı?: False
bulutlu mu?: True
mükemmel mi?: False
yağabilir mi?: True
kuru mu?: False
```

Mantıksal Operatörler

Karşılaştırma işlemlerini doğru mu (True) veya yanlış mı (False) diye birleştirmede kullanılır.

Sembol	Anlam
&&	AND - ve
||	OR - veya
!	NOT - değil

a	b	a && b	a || b
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

Mantıksal Operatörler – örnek-1

Varsayalım;

a bir `int` değişken ve değeri 7,
c bir `char` değişken ve değeri ‘r’

İfade	Sonuç
`(a >= 6) && ( c == 'r')`	True
`(a >= 6) || ( c == 'A')`	True
`(a >= 6) && ( c == 'A')`	False

Mantıksal Operatörler – örnek-2

```
int sicaklik = 75;
double yagmur = 0.35;

Console.WriteLine("sıcak?:" + (sicaklik > 70 && sicaklik < 85));
Console.WriteLine("güzel?:" + (sicaklik > 70 && yagmur < 0.4));
Console.WriteLine("çok sıcak/bulutlu?:" + (sicaklik < 50 || sicaklik >85));
Console.WriteLine("rüzgarlı?:" + (yagmur > 0.3 && yagmur < 0.7));
```

```
sıcak?: True
güzel?: True
çok sıcak/bulutlu?: False
rüzgarli?: True
```

Öncelik Kuralları (2)

- Tüm operatörler için öncelik kuralları

Öncelik	Operator
1	-, ++, --
2	*, /, %
3	+, -
4	>, >=, <, <=
5	==, !=
6	&&, ||
7	=, +=, -=, ..

- Hatırlayalım, ifadeleri parantez ile ayırmak en iyi yöntemdir.

İfadeler-Örnek(1)

```
int a = 10, b = 22;
int sonuc = a + b * 3;
Console.Write("a: {0}, b: {1}, <sonuc=a+b*3>: {2}", a, b, sonuc);
```

```
a: 10, b: 22, <sonuc=a+b*3>: 76
```

```
int a = 10, b = 22;
int sonuc = (a + b) * 3;
Console.Write("a: {0}, b: {1}, <sonuc=(a+b)*3>: {2}", a, b, sonuc);
```

```
a: 10, b: 22, <sonuc=(a+b)*3>: 96
```

İfadeler-Örnek(2)

```
int a = 7, b = 9, c = 4;
int sonuc = (a + 10) % b / c;
Console.Write("a: {0}, b: {1}, c: {2}, <sonuc=(a+10)%b/c>: {3}",
              a, b, c, sonuc);
```

```
a: 7, b: 9, c: 4, <sonuc=(a+10)%b/c>: 2
```

```
int a = 7, b = 9, c = 4;
int sonuc = (a + 10) % (b / c);
Console.Write("a: {0}, b: {1}, c: {2}, <sonuc=(a+10)%(b/c)>: {3}",
            a, b, c, sonuc);
```

```
a: 7, b: 9, c: 4, <sonuc=(a+10)%(b/c)>: 1
```

İfadeler-Örnek(3)

```
int a = 7, b = 8;
Console.Write("a(önce): {0}, ", a);
a *= b + 1;
Console.Write("a(sonra): {0}, b: {1}", a, b);
```

```
a(önce): 7, a(sonra): 63, b: 8
```

```
a = b = c = 1;
Console.Write("a(önce): {0}, b(önce): {1}, ", a, b);
a += b += c;
Console.Write("a(sonra): {0}, b(sonra): {1}, c: {2}", a, b, c);
```

```
a(önce): 1, b(önce): 1, a(sonra): 3, b(sonra): 2, c: 1
```

İfadeler-Örnek(4)

```
a = 10; b = 5;
Console.Write("a(önce): {0}, b(önce): {1}, ", a, b);
sonuc = (++a + ++b);
Console.Write("a(sonra): {0}, b(sonra): {1}, sonuc: {2}", a,b,sonuc);
```

```
a(önce): 10, b(önce): 5, a(sonra): 11, b(sonra): 6, sonuc: 17
```

```
a = 10; b = 5;
Console.Write("a(önce): {0}, b(önce): {1}, ", a, b);
sonuc = (a++ + ++b);
Console.Write("a(sonra): {0}, b(sonra): {1}, sonuc: {2}",a,b, sonuc);
```

```
a(önce): 10, b(önce): 5, a(sonra): 11, b(sonra): 6, sonuc: 16
```

Örnek Program (1)



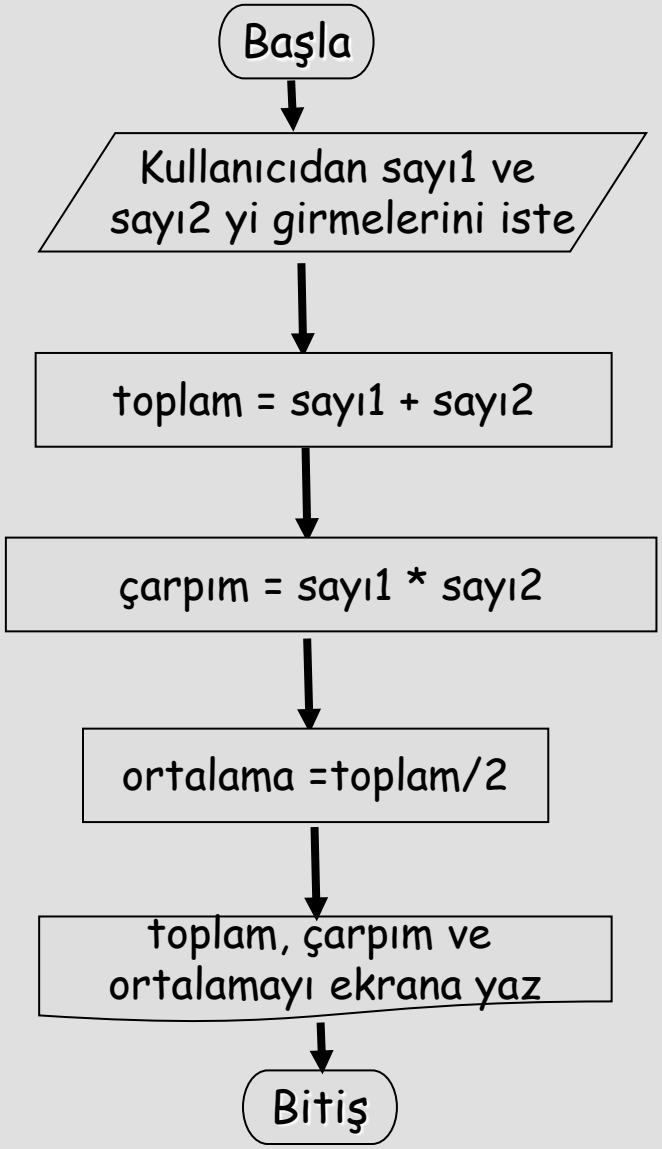

```
using System;
class Program
{
  /* Fahrenheit'ı santigrata dönüştür */
  static void Main()
  {
    double fahrenheit, santigrat;

    Console.Write("fahrenheit sıcaklığı gir: ");
    double.TryParse(Console.ReadLine(), out fahrenheit);

    santigrat = (fahrenheit - 32) / 1.8;

    Console.Write("{0} fahrenheit derece " +
        " {1:F2} santigrat dereceye eşittir",
         fahrenheit, santigrat);
  }
}
```

Örnek Program(2)



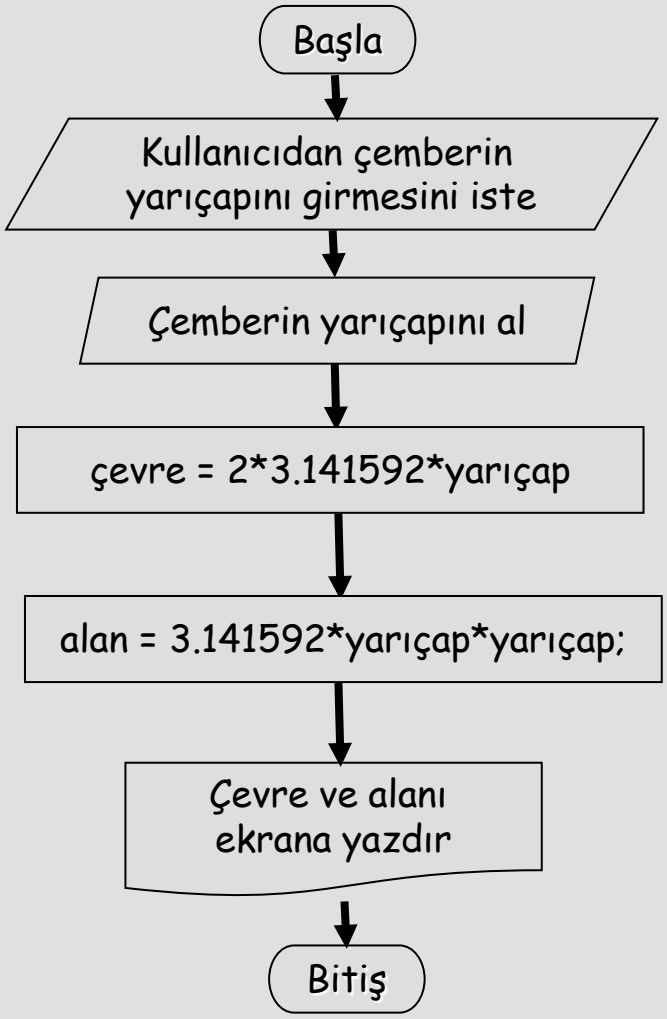

```
using System;
class Program
{
  static void Main()
  {
    Console.Write("sayı 1'i gir:");
    int sayı1 = int.Parse(Console.ReadLine());

    Console.Write("sayı 2'yi gir:");
    int sayı2 = int.Parse(Console.ReadLine());

    int toplam = sayı1 + sayı2;
    int çarpım = sayı1 * sayı2;
    float ortalama = toplam / 2;

    Console.WriteLine(
   "toplam={0} çarpım={1}, ortalama={2} "
         , toplam, çarpım, ortalama);
   }
}
```

Örnek Program(3)




```
using System;
class Program
{
  /* çemberin çevresi ve alanını hesaplama   */
  static void Main()
  {
    double yaricap, cevre, alan;
    Console.Write("yaricap girin: ");
    double.TryParse(Console.ReadLine(), out yaricap);

    cevre = 2 * 3.141592 * yaricap;
    alan = 3.141592 * yaricap * yaricap;

    Console.Write("cevre: {0:F2}, alan: {1:F2}",
                  cevre, alan);
  }
}
```

Operatörler

Birinci öncelikliler	x++, x--
Tek operand alan operatörler	+, -, !, ~, ++x, --x, (Tür)x
Çarpma ve bölme	*, /, %
Toplama ve çıkarma	+, -
Kaydırma operatörleri	<<, >>
İlişkisel ve tür testi operatörleri	<, >, <=, >=, is, as
Eşitlik operatörü	==, !=
Bitsel Ve (AND)	&
Bitsel Özel Veya (XOR)	^
Bitsel Veya (OR)	|
Mantıksal Ve	&&
Mantıksal Veya	||
Koşul operatörü	?:
Atama ve işlemli atama operatörleri	=, *=, /=, %=, +=, -=, <<=, >>=, &=, ^=, |=